

Mobile Application Reflection

James Cook University Cairns

Hunter Kruger-Ilingworth (14198489)

August 10, 2025

Contents

1 Self-Reflection: Fitness App Development 2

1.1 Description 2

1.2 Feelings 2

1.3 Evaluation 2

1.4 Analysis 3

1.5 Conclusion 4

1.6 Action Plan 5

2 References 5

List of Figures

1 Gibbs’ Reflective Cycle [1] 2

2 UI Prototypes made using draw.io 3

3 Diagram of relationship classes in the music streaming app example [2] 4

4 Room Database Structure, which has similar nesting properties to fig. 3 4

5 Room Database Table overview, made using draw.io 4

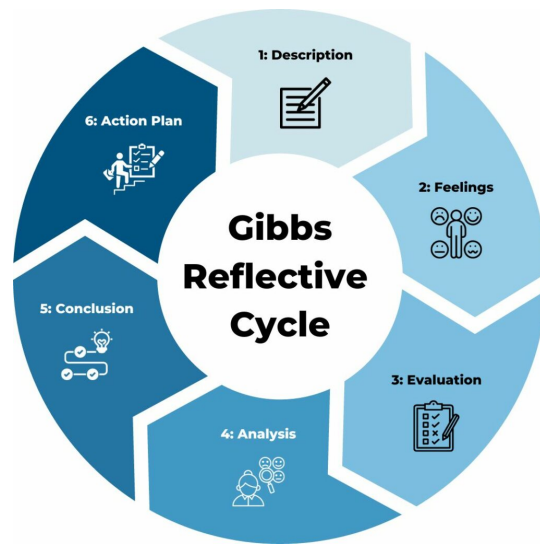


Fig. 1. Gibbs' Reflective Cycle [1]

Self-Reflection: Fitness App Development

Description

The app I developed logs sets and reps for a user during a workout. Each workout allows the user to add exercises and record individual sets, with adjustable weight units. The app's focus is a live, editable session screen, where the user checks off completed sets. A button within this screen allows the user to view historical performance data for the same exercise, offering feedback mid-session. Key features also included integration with Android Health Connect and local data persistence via Room.

The planning process for this app involved making prototype user interface (one screen) using jetpack compose. In hindsight, what would have benefitted the planning process more would have been to create lower fidelity prototype user interface separate from jetpack compose, rather than relying on the task sheets specification to do it in jetpack compose. This would have allowed me to focus on what screens I needed, and have a better understanding of what data was needed to be stored in each screen. Whilst I did this, it was further into the process than would have been optimal, after some attempts at each screen had already been made. Figure 2 shows the UI prototypes that were created during the planning phase. It was evident that these are mostly adherant to how the application ended up, but other screens did get added onto this plan, such as a settings screen, and also the exercise list selection screen, which is the dialogue which comes up whenever the user wants to add an exercise to a workout. Despite these changes, this plan was highly useful in organising my thoughts on what I wanted to make.

Feelings

Initially, I felt optimistic about integrating modern Android APIs. I believed Health Connect would be a high-value addition. As the project progressed, however, I felt frustration due to mismatched assumptions, especially when the planned API integration proved more limited than expected. Working with Room also proved more difficult than anticipated due to my app's nested data structure. Some relief came after resolving issues through staff consultation and early experimentation.

Evaluation

The core functionality real-time workout tracking and historical lookup was successfully implemented and felt intuitive. However, I encountered major issues with Health Connect integration. Although my planning

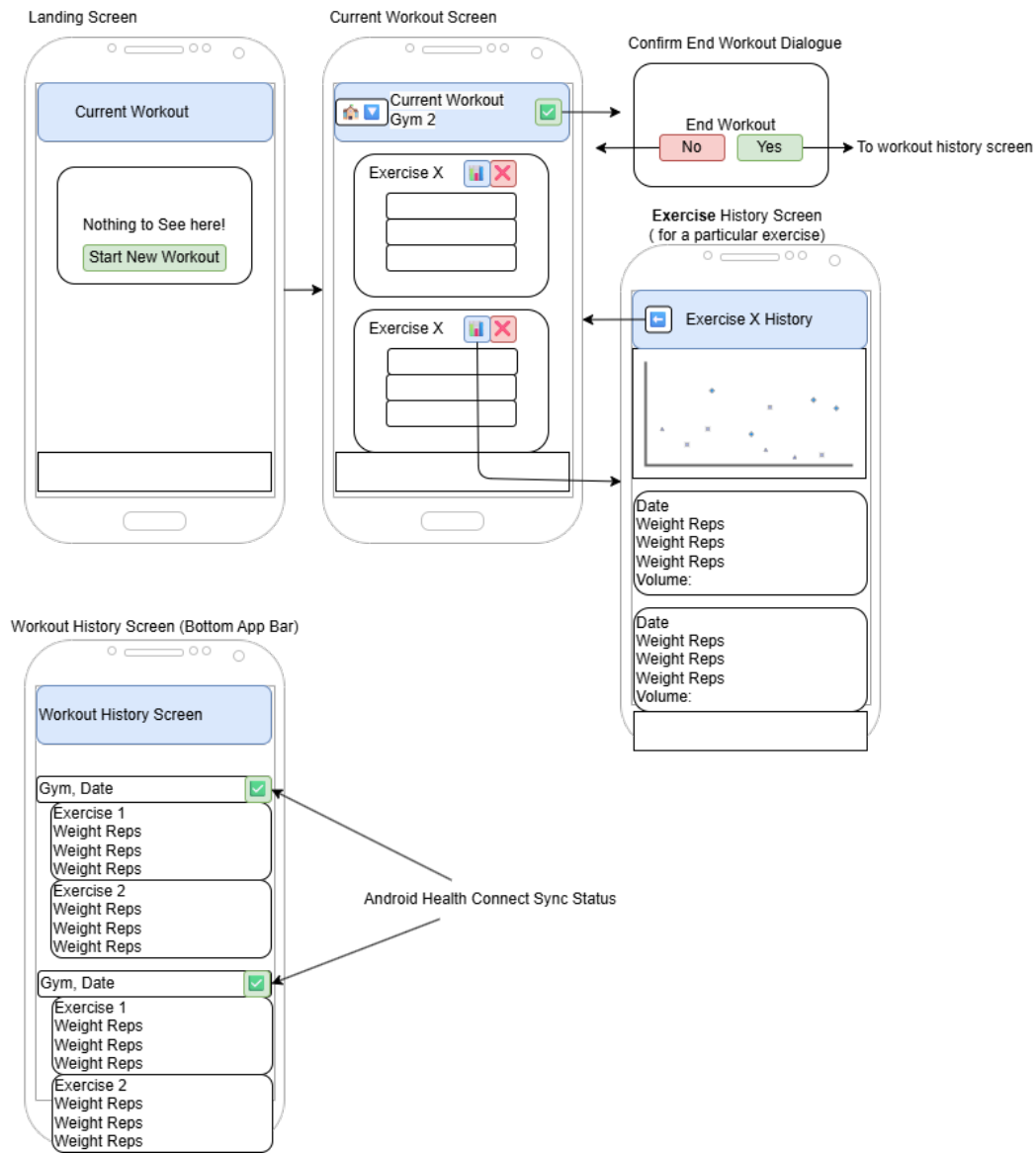


Fig. 2. UI Prototypes made using draw.io

document proposed full two-way syncing with Android Health Connect, the actual API only supports limited data types and structure mostly tailored for general activity tracking rather than gym workouts. Room integration also took considerably longer than expected due to the complexity of my relational data model.

In addition to achieving the core app functionality, I was satisfied with my use of version control throughout the project. Frequent, descriptive commits allowed me to track progress and revert changes when needed, and I made consistent use of feature branches to avoid direct pushes to `main`. I also maintained a clear and descriptive README file, outlining the key features and providing enough detail for someone to understand the application without needing to build it themselves.

Analysis

The planning phase lacked technical depth. I based my decision to use Health Connect on a surface-level reading of its documentation. In practice, it supports only basic resistance training sessions and doesn't store custom fields like gym name or weight units. The planning document also didn't make the

distinction between REST APIs (e.g., Retrofit) and Android platform APIs clear, which further added to the confusion.

Room presented additional issues. Lecture examples used flat structures (e.g., a ‘Cat’ table), which didn’t prepare me for deeply nested relationships. In my app, a workout contains groups of sets, and each group contains multiple entries, upon discussing this with Jason, we hypothesised this would be a simple foreign key type implementation, and it wouldn’t be too hard. However, implementing this in Room required indirect relationship modelling, following approaches like the *UserWithPlaylistsAndSongs* example from the Android developer documentation [2]. Without prior experience in SQL or relational modelling, this was a steep learning curve.

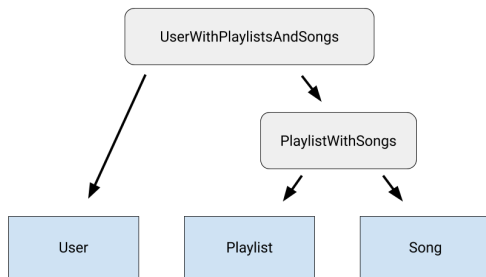


Fig. 3. Diagram of relationship classes in the music streaming app example [2]

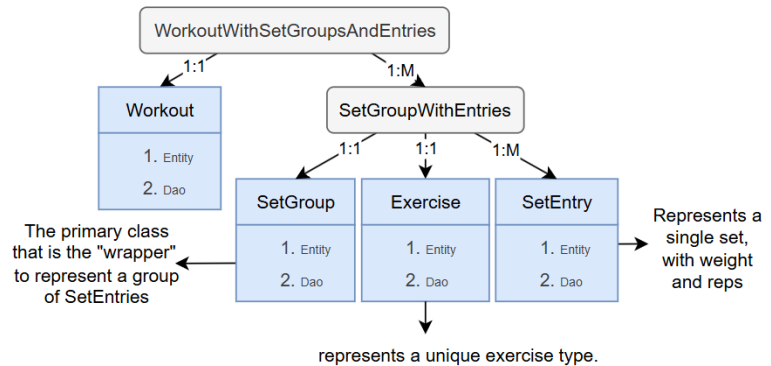


Fig. 4. Room Database Structure, which has similar nesting properties to fig. 3

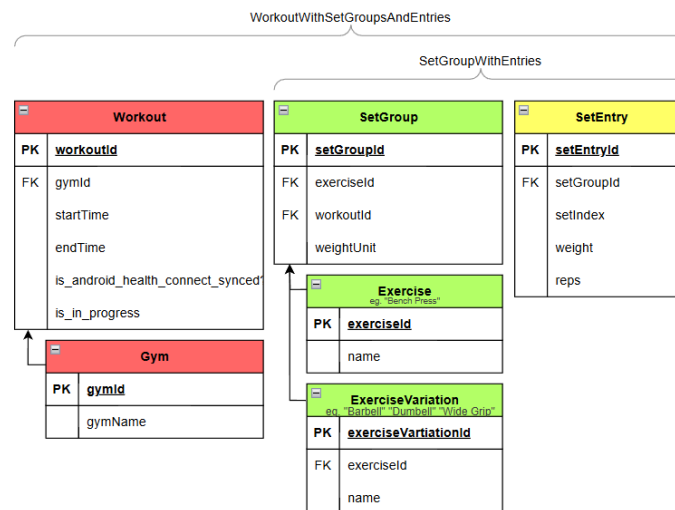


Fig. 5. Room Database Table overview, made using draw.io

Conclusion

Figure 5 shows the table structure from my planning process, where it is evident that I implemented all tables except for the ExerciseVariation table in my Room database. Including the ExerciseVariation table would have further improved the usability of the application by allowing users to track different variations of exercises more easily, rather than having to create a whole new exercise. This would have been less cumbersome, especially since users would otherwise need to create an exercise with the same name and add the variation (e.g., "Bench Press - Narrow Grip").

In retrospect, I should have validated the technical feasibility of APIs and database structures earlier. I also underestimated the complexity of Room when dealing with nested data. That said, seeking feedback early (e.g., from Jason) and prioritising the most difficult implementation tasks proved helpful. The process highlighted the importance of robust early planning and being prepared for misalignment between course structure and project needs.

Action Plan

In future projects, I will:

- Perform deeper technical reviews of APIs before committing to them in planning.
- Build early prototypes to explore feasibility of complex features.
- Engage with staff early when facing architecture decisions. This helped me a lot in this project.
- Map data relationships visually before implementing persistence layers.
- Allow more flexibility in the planning phase to adjust to technical limitations.

References

- [1] M. Channell. (2023) Gibbs reflective cycle – a practical guide: Reflect and grow: Mastering leadership skills with gibbs’ reflective cycle. [Online]. Available: <https://www.tsw.co.uk/blog/leadership-and-management/gibbs-reflective-cycle/>
- [2] A. Developers, “Diagram of relationship classes in the music streaming app example,” <https://developer.android.com/training/data-storage/room/relationships/nested>, n.d.